

PROJET - Déploiement

Analyse Getaround 🚗 840 min



GetAround, c'est un peu le Airbnb de la voiture. Vous pouvez louer une voiture auprès de n'importe quel particulier pour quelques heures ou quelques jours ! Fondée en 2009, cette entreprise a connu une croissance fulgurante. En 2019, elle comptait plus de 5 millions d'utilisateurs et environ 20 000 voitures disponibles dans le monde entier.

En tant que partenaire de Jedha, ils ont proposé de grands défis :

Contexte

Lors de la location d'une voiture, nos utilisateurs doivent effectuer une procédure d'enregistrement au début de la location et une procédure de restitution au terme de la location afin de :

- Évaluer l'état du véhicule et informer les autres parties des dommages préexistants ou des dommages survenus pendant la location.
- Comparer les niveaux de carburant.
- Mesurer le nombre de kilomètres parcourus.

L'arrivée et le départ de nos locations peuvent se faire selon trois procédures distinctes :

- 📱 **Contrat de location mobile** sur applications natives : le conducteur et le propriétaire se rencontrent et signent tous deux le contrat de location sur le smartphone du propriétaire.
- **Connexion** : le conducteur ne rencontre pas le propriétaire et ouvre la voiture avec son smartphone.
- 📄 **Contrat papier** (négligeable)

Projet 🚧

Pour cette étude de cas, nous vous suggérons de vous mettre à notre place et de reproduire l'analyse que nous avons réalisée en 2017 🌍 ✨

Lorsqu'ils utilisent Getaround, les conducteurs réservent des voitures pour une durée déterminée, allant d'une heure à quelques jours. Ils sont censés restituer le véhicule à l'heure prévue, mais il arrive parfois qu'ils soient en retard au point de restitution.

Les retours tardifs au moment de la restitution du véhicule peuvent engendrer de fortes tensions pour le conducteur suivant si la voiture devait être louée à nouveau le même jour : le service client signale souvent des utilisateurs insatisfaits car ils ont dû attendre le retour du véhicule après la location précédente, ou même des utilisateurs qui ont dû annuler leur location car la voiture n'a pas été rendue à temps.

Objectifs 🎯

Afin d'atténuer ces problèmes, nous avons décidé d'instaurer un délai minimum entre deux locations. Un véhicule ne sera pas affiché dans les résultats de recherche si les heures d'arrivée ou de départ demandées sont trop proches d'une location déjà réservée.

Cela résout le problème des départs tardifs, mais risque aussi de nuire aux revenus de Getaround et des propriétaires : nous devons trouver le juste compromis.

Notre chef de produit doit encore se décider :

- **Seuil** : quelle doit être la durée minimale du délai ?
- **Portée** : faut-il activer cette fonctionnalité pour toutes les voitures ou uniquement pour les voitures connectées ?

Afin de les aider à prendre la bonne décision, ils vous demandent de leur fournir des informations sur les données. Voici les premières analyses auxquelles ils ont pensé pour lancer la discussion. N'hésitez pas à effectuer des analyses complémentaires que vous jugerez pertinentes.

- Quelle part des revenus de notre propriétaire serait potentiellement affectée par cette fonctionnalité ?
- Combien de locations seraient concernées par cette fonctionnalité, en fonction du seuil et de la portée que nous choisirons ?
- À quelle fréquence les chauffeurs sont-ils en retard pour le prochain pointage ? Quel impact cela a-t-il sur le chauffeur suivant ?
- Combien de cas problématiques cela résoudra-t-il en fonction du seuil et de la portée choisis ?

Tableau de bord Web

Commencez par créer un tableau de bord qui aidera l'équipe de gestion des produits à répondre aux questions ci-dessus. Vous pouvez utiliser [nom de la `streamlit` technologie] ou toute autre technologie de votre choix.

Apprentissage automatique - `/predict` point d'extrémité

En plus de la question ci-dessus, l'équipe de science des données travaille sur *l'optimisation des prix* . Elle a collecté des données afin de suggérer des prix optimaux aux propriétaires de voitures grâce à l'apprentissage automatique.

Vous devez fournir au moins **un point de terminaison** `/predict` . L'URL complète ressemblerait à ceci : `https://your-url.com/predict` .

Ce point de terminaison accepte **les requêtes POST** avec des données d'entrée JSON et doit renvoyer les prédictions. Nous supposons que **les données d'entrée sont toujours correctement formatées** . Par conséquent, la gestion des erreurs est inutile. Nous laissons cette gestion à votre discrétion.

Exemple de saisie :

```
{
  "input": [[7.0, 0.27, 0.36, 20.7, 0.045, 45.0, 170.0, 1.001, 3.0, 0.45, 8.8], [7.0, 0.27, 0.36, 20.7, 0.
```

La réponse doit être un JSON avec une clé `prediction` correspondant à la prédiction.

Exemple de réponse :

```
{
  "prediction": [6,6]
}
```

page de documentation

Vous devez fournir aux utilisateurs une **documentation** sur votre API.

Il doit se trouver à l'adresse `/docs` de votre site web. Si nous reprenons l'exemple d'URL ci-dessus, il doit se trouver directement à `https://your-url.com/docs` l'adresse indiquée.

Cette brève documentation devrait au moins inclure :

- Titre h1 : le titre est à votre discrétion.
- Une description de chaque point de terminaison que l'utilisateur peut appeler, avec le nom du point de terminaison, la méthode HTTP, l'entrée requise et la sortie attendue (vous pouvez donner un exemple).

Vous pouvez ajouter toute autre information pertinente et styliser votre code HTML comme vous le souhaitez.

Production en ligne

Vous devez **héberger votre API en ligne** . Nous vous recommandons Hugging Face , car il est gratuit. Vous pouvez toutefois choisir n'importe quel autre fournisseur d'hébergement.

Aides 🐘

Pour vous aider à démarrer ce projet, voici quelques conseils :

- Consacrez du temps à comprendre les données
- Ne négligez pas la partie analyse des données, elle recèle de nombreuses informations précieuses.
- L'analyse des données devrait prendre entre 2 et 5 heures.
- L'apprentissage automatique devrait prendre entre 3 et 6 heures.
- Vous n'êtes pas obligé d'utiliser des bibliothèques pour gérer votre flux de travail d'apprentissage automatique, `mlflow` mais nous vous le conseillons vivement.

Partagez votre code

Pour que votre code soit évalué, n'oubliez pas de le partager sur un dépôt [GitHub](#) . Vous pouvez créer un `README.md` fichier contenant une brève description du projet, la procédure d'installation locale et l'URL en ligne.

Livrable 📁

Pour mener à bien ce projet, vous devez livrer :

- Un **tableau de bord** en production (accessible via une page web par exemple)
- L' **intégralité du code** est stockée dans un **dépôt Github** . Vous devrez inclure l'URL de ce dépôt.
- Une **API en ligne documentée** sur le serveur Hugging Face (ou tout autre fournisseur de votre choix) contenant au moins un `/predict` point de terminaison conforme à la description technique ci-dessus. Nous devrions pouvoir interroger ce point de terminaison de l'API `/predict` en utilisant `curl` :

```
$ curl -i -H "Content-Type: application/json" -X POST -d '{"input": [[7.0, 0.27, 0.36, 20.7,
```

Ou Python :

```
import requests

response = requests.post("https://your-url/predict", json={
    "input": [[7.0, 0.27, 0.36, 20.7, 0.045, 45.0, 170.0, 1.001, 3.0, 0.45, 8.8]]
})
print(response.json())
```

Données

Vous devez télécharger deux fichiers :

- Analyse des retards 👉 Analyse des données
- Optimisation des prix 👉 Apprentissage automatique